MDQ Specification Audit: essay: mdq-spec

Date: 2026-10-20

Scope: The essay question type: `docs/question-types/essay.md` and `schema/essay.yaml`, read against `docs/references/grammar.md`, `docs/references/patterns.md`, `docs/question-types/question-base.md`, `docs/exam.md` and `docs/responses.md`. Earlier audits already settled the shared rules.

Methodology

The audit script `scripts/spec_audit.py` passed all 15 steps before the audit started: it compiles every Lark snippet, checks the Additional Rules tables against `docs/lint-codes.md` and the corpus, and checks the frontmatter tables against the schemas.

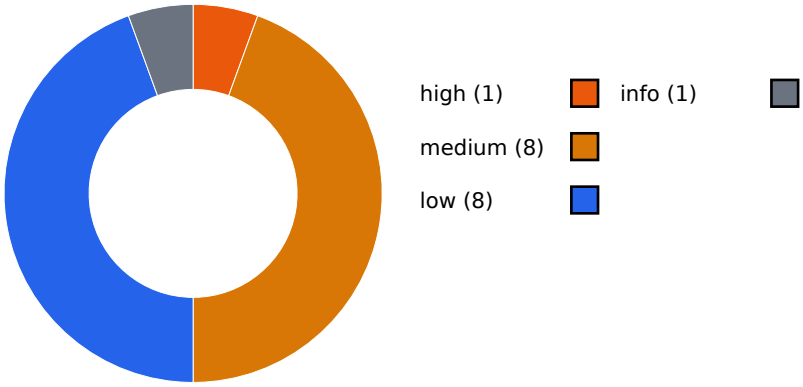
The lead auditor read `docs/question-types/essay.md` with the shared specs and recorded contradictions, ambiguities, omissions and inconsistencies. The audit focused on what is specific to essay: the `[essay]` tag, the `input` and `highlight` fields, the answer key section, the response and the manual grading. Edge cases were run through mdq-py to see how the reference implementation reads the text.

One subagent turned the essay grammar and the shared grammar into Lark and tested what they accept and reject. A second, independent subagent scanned the essay document and schema, ran each complete example through `mdq validate`, and checked the Additional Rules table against the MUST, SHOULD and MAY requirements of the text.

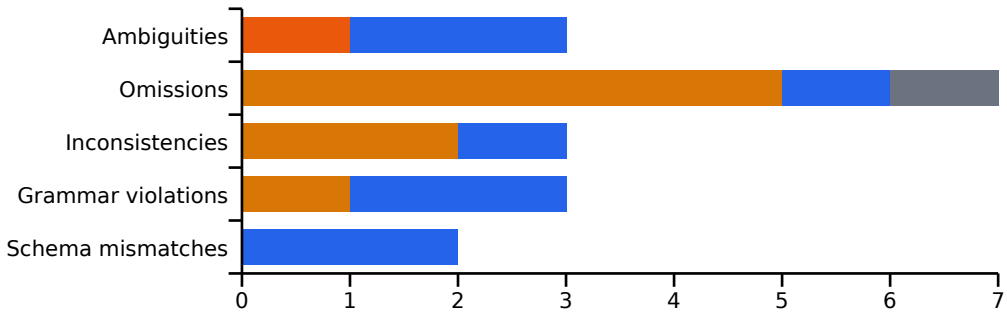
Executive summary

Severity	#
CRITICAL	0
HIGH	1
MEDIUM	8
LOW	8
INFO	1
Total findings	**18**

Findings by severity



Findings by category



Strengths and weaknesses

Strengths

✓ **Grammar violations:** The essay tag literal "`[essay]`" agrees with the bracketed-tag rules of question-base.md:211-217: tested with Lark, it accepts `[essay]` and rejects `[ essay ]` (whitespace inside brackets), `[Essay]` (case), `[essay] write here`, `[essay]\nwrite here` (more than a single tag) and `[essay](http://x)` (a link). mdq-py gives the same result for each of these inputs. (docs/question-types/essay.md:34-36)

✓ **Omissions:** The essay corpus covers the tag variants and misplacements: spaced, uppercase, NBSP-indented, indented, in a list, in a blockquote, with trailing text, repeated, and an answer key before the body or repeated. (examples/invalid/essay-spaced-tag.mdq.md, examples/invalid/essay-tag-uppercase.mdq.md, examples/invalid/essay-tag-in-list.mdq.md, examples/invalid/essay-two-tags.mdq.md, examples/invalid/essay-two-answer-keys.mdq.md, examples/valid/essay/indented-tag.mdq.md)

✓ **Schema mismatches:** schema/essay.yaml agrees with essay.md on the `input` enum and its default `text`, on `highlight` being ignored with a warning for non-code input, and on `answerKey` blank-text. (schema/essay.yaml:16-48)

✓ **Inconsistencies:** The essay lint codes in essay.md agree with docs/lint-codes.md in name, level and scope (`ignored-highlight`, `unknown-highlight-language`, `code-input-without-highlight`, `missing-answer-key`, `foreign-choice-marker`). (docs/lint-codes.md:17-80)

✓ **Contradictions:** Both complete examples in essay.md (lines 7-11 and 63-78) validate with mdq-py with no error or warning. (docs/question-types/essay.md:7-78)

Central risks

• Essay looks like the simplest type, but most of its rules live in the corpus and in mdq-py, not in the text: the tag paragraph, the answer key section, its extent and its empty form.

• The response of a rich text essay has no defined format, so two hosts can exchange the same answer as Markdown and as HTML.

Detailed findings

Contradictions

No findings in this category.

Ambiguities

Severity	Location	Description
HIGH	docs/question-types/essay.md:47-50, docs/responses.md:38	**F5** The response format of a "text" essay is not defined
LOW	docs/question-types/essay.md:55-57	**F12** When the answer key may be shown to students is not defined
LOW	docs/question-types/essay.md:55, docs/question-types/essay.md:82	**F20** Essay spec says both that essays may require and that they always use manual grading

F5: The response format of a "text" essay is not defined

Location: docs/question-types/essay.md:47-50, docs/responses.md:38

```
"text" indicates that the response should be a rich text input, potentially
supporting markdown or other formatting.
```

Description: `input: text` asks for a rich text editor "potentially supporting markdown or other formatting", and `docs/responses.md:38` says the essay response is "the text the student wrote". Neither document says what the response string holds for a rich text editor: Markdown source, HTML, or text with the formatting removed. `input: code` and `input: plain` have no such doubt.

Impact: Two hosts that exchange responses (an LMS, a grader, an exporter) can store the same answer as Markdown in one and as HTML in the other. A renderer cannot tell which it received, and raw HTML in a response is also an injection risk when the host renders it as Markdown.

Suggested fix: State in `docs/responses.md` and in Input that the response of every essay is a string, and that for `input: text` it is CommonMark source. A host whose editor produces HTML converts it before it stores the response. `plain` and `code` responses are verbatim text.

F12: When the answer key may be shown to students is not defined

Location: docs/question-types/essay.md:55-57

```
key section is optional and serves as a guide for the instructor to grade the
responses and may be presented as feedback to the students.
```

Description: The answer key "may be presented as feedback", but the text does not say when (after the response, after grading, after the exam closes) or whether a host must hide it before. The other types define feedback that shows after grading; a pending essay "shows no feedback" (`responses.md:319-320`).

Impact: A host can show the model answer while the exam is open, and the rule that a pending response shows no feedback seems to forbid showing it at all until the instructor grades.

Suggested fix: State that the answer key MUST NOT be shown to the student before they submit the response, and that it MAY be shown after, even while the response is pending.

F20: Essay spec says both that essays may require and that they always use manual grading

Location: docs/question-types/essay.md:55, docs/question-types/essay.md:82

Mechanical fix

```
Essay questions are usually open-ended, and may require manual grading.
```

Description: Line 55 says essays "may require manual grading", which implies some are graded automatically. Line 82 and question-base.md:280 say essays are always graded manually (`manual` automation).

Impact: A reader can expect some automatic grading of essays, for example from the answer key.

Suggested fix: Replace "may require manual grading" with a statement that every essay response is graded by the instructor.

Omissions

Severity	Location	Description
MEDIUM	docs/question-types/essay.md:59-61	**F3** The answer key section has no grammar, and its heading variants and extent are not specified
MEDIUM	docs/question-types/essay.md:92-95	**F7** An empty or repeated answer key section has no documented rule

Severity	Location	Description
MEDIUM	docs/question-types/essay.md:53-78	**F8** The text never maps the answer key section to the answerKey field
MEDIUM	docs/question-types/essay.md:80-82	**F9** The Grading section does not give the score range, automation or response
MEDIUM	docs/exam.md:161-165	**F25** Exam rules about `---` and setext headings do not cover the essay answer key
LOW	docs/question-types/essay.md:59-61	**F26** Essay spec does not say how footnotes resolve across the stem, epilogue and answer key
INFO	docs/question-types/essay.md:15-22	**F13** No fields for response size or attachments

F3: The answer key section has no grammar, and its heading variants and extent are not specified

Location: docs/question-types/essay.md:59-61

```
It starts with an `H2` heading with the text `## [answer-key]`
```

Description: The answer key section is defined only in prose; essay.md has no rule for it, unlike the `## [extra]` section of ordering (ordering.md:146: `"##" ws "[" "extra" "]" nl+ content`). The prose says both "an H2 heading" and "the text `## [answer-key]`", so it does not settle these inputs: the setext H2 `[answer-key]` + `---` (mdq-py accepts it as the answer key), `## [answer-key]` (mdq-py accepts), `## [Answer-Key]`, `### [answer-key]` and `## [answer-key] Notes` (mdq-py rejects them as `forbidden-block-element` in the epilogue). The prose also does not say where the section ends or what blocks it may hold: mdq-py reads every block up to the end of the question, so a later `## Rubric` heading and its list are inside answerKey. Inside an exam the end is the next `===` or `---` block, which the text does not mention. The forbidden elements of question-base.md are not checked in the section either: mdq-py keeps an H1 and a second `[essay]` paragraph after the answer key as answer key text (`answerKey: "Key.\n\n[essay]"`), and prose after the answer key meant as epilogue goes into it. No example in examples/ has a heading, a bracketed paragraph or more prose after the answer key, or any heading form other than `## [answer-key]`.

Impact: Two implementations can split the same document into a different epilogue and answer key.

Suggested fix: Add a grammar rule, for example `answer_key : "##" ws "[" "answer-key" "]" ws? nl md_block+`. State that the heading is an ATX H2 alone on its line (decide on the setext form; CommonMark reads both as H2), and that the section holds every block after it, headings included except H1, up to the end of the question (end of file, or the next block in an exam).

F7: An empty or repeated answer key section has no documented rule

Location: docs/question-types/essay.md:92-95

```
| answerKey | error    | blank-text                  | if defined, must have at least one
visible character      |
```

Description: The Additional Rules table has no rule for a second `## [answer-key]` section, but `examples/invalid/essay-two-answer-keys.mdq.md` expects a `parse-error`. For a section with no content (`## [answer-key]` as the last line), the `blank-text` rule suggests an error, but mdq-py accepts the document and drops the field: the result has no answerKey and no error. The corpus already decides the empty case the other way: examples/valid/essay/empty-answer-key.mdq.md is valid and expects only `missing-answer-key`, while examples/invalid/model-only/essay-blank-answer-key.yaml is a `blank-text` error. So the rule that an empty Markdown section defines no answer key exists only in the corpus.

Impact: The corpus enforces a rule the spec does not state, and an empty section, which is almost always an unfinished question, passes with only the `missing-answer-key` info at most.

Suggested fix: Add a rule for a second section (`parse-error`) to the table. For the empty section, two readings are defensible: (a) an empty section is an absent answer key, reported only as `missing-answer-key`, as the corpus decides (examples/valid/essay/empty-answer-key.mdq.md); (b) it is a defined but blank field, `blank-text`, as for `title:` with only spaces. Recommended: (a), since `## [answer-key]` with nothing after it is a common template placeholder, and state it in the text and the table.

F8: The text never maps the answer key section to the `answerKey` field

Location: docs/question-types/essay.md:53-78

```
## Answer key
```

Description: The Answer key section describes the Markdown syntax but never states that its content becomes the answerKey field of the AST, nor how the blocks are joined into the string. The field name appears only in the Additional Rules table (lines 92 and 95) and in `schema/essay.yaml:41`. The document also has no YAML form of a parsed essay.

Impact: A reader of the Markdown spec cannot tell which field holds the answer key, or that it is a field of the body and therefore invalid in the frontmatter (`invalid-frontmatter-field`).

Suggested fix: Add one sentence: the blocks of the section, without the heading, are the answerKey string, with the same normalization license as `stem` and epilogue. Add the YAML form of the example.

F9: The Grading section does not give the score range, automation or response

Location: docs/question-types/essay.md:80-82

```
Essay questions are graded manually.
```

Description: The rules that apply to an essay are spread over other documents: the automation is `manual` (`question-base.md:280`), every response is pending (`responses.md:312-313`), the score is in [0, 1] (`exam.md:359`), the exam `grading` does not apply (`exam.md:359`), and the response is a string (`responses.md:190-204`). The essay document links to none of them, and the instructor's score range is stated only for fill-in blanks (`responses.md:320-321`). `ordering.md` has Automation and Response sections; essay does not.

Impact: A reader of the essay document cannot learn how an essay is scored or what its response looks like without reading three other documents.

Suggested fix: Expand Grading: the automation is `manual`, every response is pending until the instructor gives a score in [0, 1], the exam `grading` does not apply and `penalty` does. Add a Response section that links to `responses.md`.

F25: Exam rules about `---` and setext headings do not cover the essay answer key

Location: docs/exam.md:161-165

```
Inside an exam, the epilogue of a question MUST NOT use `---` as a thematic
break.
```

Description: The exam rules name only the epilogue (and the stem and instructions for `setext-heading`). The answer key comes after the epilogue and is the last block of an essay in an exam, so a `---` in it has the same problem. mdq-py applies the epilogue rule to it: a `---` in the answer key gives `parse-error` with the message "a question's epilogue cannot use '---'", and a setext `[answer-key]` heading gives `setext-heading` at path `epilogue` while it parses as `answerKey`. The spec does not state either behavior and no example in examples/ covers it.

Impact: Implementations can accept a `---` in an answer key, and a later question then starts inside it.

Suggested fix: Extend the exam rules to every block after the body, the answer key included.

Exploitability conditions: An essay with an answer key inside an exam.

F26: Essay spec does not say how footnotes resolve across the stem, epilogue and answer key

Location: docs/question-types/essay.md:59-61

```
The answer key section is placed below the `[essay]` tag and any existing
```

Description: Each part is stored as its own Markdown string. mdq-py accepts a stem with `[^1]` whose definition is in the answer key: the stem keeps a reference with no definition, and the definition is hidden with the answer key. The spec does not say if a footnote reference must be defined in the same field, and no example in examples/ covers it (examples/valid/essay/epilogue-starts-with-footnote.mdq.md only has an epilogue footnote).

Impact: The student view renders a dangling footnote, or a host that renders the whole document shows text from the answer key.

Suggested fix: State that a footnote definition belongs to the field it is in, and lint a reference defined in another field.

Exploitability conditions: An essay that uses footnotes and has an answer key.

F13: No fields for response size or attachments

Location: docs/question-types/essay.md:15-22

```
| input     | string  | Either "code", "text" or "plain"[^2]      |
```

Description: Essay has no field for a minimum or maximum length (words or characters), the size of the input box, or file attachments. Moodle essay questions have `responsefieldlines`, `minwordlimit`, `maxwordlimit` and `attachments`; the `mdq export` targets lose nothing today, but an import from Moodle drops them.

Impact: Authors write length limits in the stem (`essay-tag-with-text` shows the attempt), where no host can enforce them.

Suggested fix: Decide whether MDQ covers these. If yes, add optional `minWords`/`maxWords` fields; if no, state that limits belong in the stem or in `meta`.

Inconsistencies

Severity	Location	Description
MEDIUM	docs/question-types/essay.md:59-60, docs/question-types/ordering.md:166-168	**F21** Essay places its answer key section after the epilogue, ordering places its sections before it

Severity	Location	Description
MEDIUM	docs/question-types/essay.md:21, docs/question-types/ordering.md:98	**F22** Essay `input` and ordering `content` hold the same code/text choice under different names
LOW	docs/question-types/essay.md:24, docs/question-types/essay.md:91-94, docs/question-types/ordering.md:339	**F11** A warning rule is written as "must"

F21: Essay places its answer key section after the epilogue, ordering places its sections before it

Location: docs/question-types/essay.md:59-60, docs/question-types/ordering.md:166-168

```
The answer key section is placed below the `[essay]` tag and any existing
epilogue block.
```

Description: Both types use `## [name]` sections after the body. In essay the epilogue comes before `## [answer-key]` (examples/valid/essay/epilogue-and-answer-key.mdq.md). In ordering the sections follow the content block directly and the epilogue comes after the last section; prose between the block and a section is a `parse-error`. question-base.md:4-6 and 264-265 describe the essay order (special sections after the epilogue), so ordering disagrees with the base, and the two types disagree with each other.

Impact: Authors must learn two placements for the same `## [name]` construct, and a parser cannot share the section logic.

Suggested fix: Choose one placement for `## [name]` sections relative to the epilogue and use it in question-base.md, essay.md and ordering.md.

Exploitability conditions: An essay or ordering question with both an epilogue and a section.

F22: Essay `input` and ordering `content` hold the same code/text choice under different names

Location: docs/question-types/essay.md:21, docs/question-types/ordering.md:98

```
| input     | string  | Either "code", "text" or "plain"[^2]      |
```

Description: Both fields select between code (paired with `highlight`, with the same `ignored-highlight` and `unknown-highlight-language` rules) and `text`. Essay calls the field `input` and adds `plain`; ordering calls it `content` and has no `plain`. The lint-codes.md row of `ignored-highlight` has to say "`input`/`content`".

Impact: Authors who write both types must remember two names for one concept.

Suggested fix: Use one field name for the code/text choice in both types, or document why they differ.

F11: A warning rule is written as "must"

Location: docs/question-types/essay.md:24, docs/question-types/essay.md:91-94, docs/question-types/ordering.md:339

Mechanical fix

```
| highlight | warning  | ignored-highlight           | must be omitted unless `input` is "code"                  |
```

Description: The level is `warning`, which maps to SHOULD, but the rule says "must be omitted". Footnote 1 says only that the property is ignored, with no requirement level. `question-base.md` writes its warnings as "should" (`unsafe-id`: "should be url-safe"). The info rule `code-input-without-highlight` (essay.md:94) is not described in the prose either.

Impact: The table contradicts its own level, and a reader can take the rule as an error.

Suggested fix: Write "should be omitted unless ..." in both tables, and in footnote 1 add that the property SHOULD be omitted.

Grammar violations

Severity	Location	Description
MEDIUM	docs/question-types/essay.md:35	**F1** Essay grammar rejects an indented [essay] tag and trailing spaces
LOW	docs/question-types/essay.md:30-36	**F2** Essay grammar does not say it applies to a paragraph, so it accepts the content of a list item
LOW	docs/question-types/question-base.md:194	**F4** SLUG accepts essay, so `[essay]` at the start of the first paragraph is a slug

F1: Essay grammar rejects an indented [essay] tag and trailing spaces

Location: docs/question-types/essay.md:35

Mechanical fix

```
essay : "[essay]"
```

Description: The rule has no `ws?` before or after the tag. Tested with Lark: `[essay]`, `[essay]`, `\t[essay]` and `[essay]  ` are all rejected. But question-base.md:219-220 says "a tag may be indented", and grammar.md:59-69 defines the leading `ws?` of a block as the CommonMark indentation of at most three spaces. The numeric and short-answer tag rules (numeric.md:75, short-answer.md:154) start with `ws?` and end with `ws?`; the essay rule does not. mdq-py accepts `[essay]`, `[essay]` and `[essay]  ` as essay bodies, and rejects `[essay]` (code block), which matches the prose, not the grammar. The `ordering` rule (ordering.md:141) has the same gap; the ordering audit covers it.

Impact: A reader who follows the grammar rejects valid documents with an indented tag or trailing spaces; an implementation built from the grammar disagrees with mdq-py.

Suggested fix: Write the rule as `essay : ws? "[essay]" ws?`, as the numeric and short-answer rules do.

F2: Essay grammar does not say it applies to a paragraph, so it accepts the content of a list item

Location: docs/question-types/essay.md:30-36

```
The body consists of a single `[essay]` tag.
```

Description: The grammar matches only the text `[essay]` and does not state the block that holds it. grammar.md:73-79 says a list-item grammar applies to the item content after the removal of the marker. With that reading, the content of `* [essay]` is `[essay]` and the grammar accepts it, while the Additional Rules table (essay.md:90) forbids the tag as a list item (`foreign-choice-marker`). The content of the heading `## [essay]` is also `[essay]`. mdq-py reads only a paragraph as the tag: `* [essay]` gives `foreign-choice-marker` and `## [essay]` gives a missing body. The grammar also does not say that the tag is the whole paragraph: `Describe the Pantanal.\n[essay]` (one paragraph) is rejected by mdq-py, and the spec settles this only by the word "single". The corpus fixes more cases that the Additional Rules table does not list: `essay-tag-with-text` (`[essay] com no máximo 200 palavras`) and `essay-tag-uppercase` (`[Essay]`) expect `parse-error`, and `essay-two-tags` expects `forbidden-block-element` in the epilogue.

Impact: The block context of the tag comes from implementation behavior and a footnote, not from the grammar.

Suggested fix: State that the body is a paragraph whose only content is the rule, for example `essay_block : ws? "[essay]" ws? nl`, and that a list item or heading is not the tag. State that a later `[essay]` paragraph is a forbidden element of the epilogue, and list the parse errors in the Additional Rules table, as other types do.

F4: SLUG accepts `essay`, so `[essay]` at the start of the first paragraph is a slug

Location: docs/question-types/question-base.md:194

```
SLUG: /[a-zA-Z0-9]+([-_][a-zA-Z0-9]+)*/
```

Description: The slug rule (question-base.md:174-175, 197-199) reads a bracket at the start of the first paragraph as a slug when its content matches SLUG, and `essay` matches. No rule excludes the type tag names. So `[essay] Describe the Pantanal.` followed by `[essay]` is a question with id `essay` (mdq-py output: `{"type": "essay", "id": "essay", ...}`). A document whose first paragraph is only `[essay]` is either a slug-only stem paragraph or the body with no stem; the spec does not say which (mdq-py reads it as the body and reports a missing field).

Impact: Contrived inputs where the spec gives two readings of the same bracket.

Suggested fix: State whether a type tag name such as `essay` can be a slug, and that a paragraph of only `[essay]` is always the essay tag.

Schema mismatches

Severity	Location	Description
LOW	schema/essay.yaml:32-33	**F28** schema/essay.yaml gives `highlight` a validation role the spec does not have
LOW	schema/essay.yaml:35	**F29** schema/essay.yaml rejects an empty `highlight`, the spec does not say so

F28: schema/essay.yaml gives `highlight` a validation role the spec does not have

Location: schema/essay.yaml:32-33

Mechanical fix

```
Programming language used for syntax highlighting and, where
          supported, validation.
```

Description: The schema says `highlight` is used for syntax highlighting "and, where supported, validation". essay.md:44-45 says only that it specifies the language for syntax highlighting. The spec does not define any validation of the response by language.

Impact: Hosts can reject responses that do not compile, which the spec does not permit or describe.

Suggested fix: Remove the validation clause from the schema, or describe the behavior in essay.md.

F29: schema/essay.yaml rejects an empty `highlight`, the spec does not say so

Location: schema/essay.yaml:35

Mechanical fix

```
minLength: 1
```

Description: The schema sets `minLength: 1` on `highlight`, so `highlight: ""` is a `schema-error` (mdq-py confirms). essay.md does not state this constraint; question-base.md:366-367 says identifiers and syntax fields reject the empty string, but essay.md does not say that `highlight` is one.

Impact: Readers of essay.md cannot tell that an empty `highlight` is an error and not an ignored value.

Suggested fix: Add the constraint to the essay frontmatter table or footnote [^1].

Prioritized recommendations

P1: Define the `[essay]` paragraph and the answer key section in the grammar and the Additional Rules table. (Findings: F2, F3, F7, F8, F4, F26)

P1: Define the response format of each essay `input`. (Findings: F5)

P1: Apply the mechanical fixes: `ws?` in the tag rule, "should" in the warning rules, the grading sentence of line 55, and the two `highlight` schema descriptions. (Findings: F1, F11, F20, F28, F29)

P2: Use one placement for `## [name]` sections in essay and ordering, and extend the exam `---` rules to them. (Findings: F21, F25)

P2: Document essay grading, response and answer key visibility in essay.md. (Findings: F9, F12, F13)

P3: Explain or unify essay `input` and ordering `content`. (Findings: F22)

GitHub issues

Each block below is a complete issue, ready to paste.

````
--- ISSUE 1 ---
# [Spec] Define the `[essay]` paragraph and the `## [answer-key]` section in the grammar

**Labels:** medium, low

## Problem

The essay grammar is one literal, and the answer key section has no grammar at all. What the
tag paragraph is, how the answer key heading is spelled, where the section ends, what an empty
or repeated section is, and which field it fills are known only from the corpus and mdq-py.
Recommended: write `essay_block` and `answer_key` rules, state that the section runs to the end
 of the question and fills `answerKey`, document an empty section as an absent answer key
(corpus behavior), a second section as `parse-error`, and that a footnote belongs to the field
it is in. Decide whether a setext `[answer-key]` heading is valid (recommended: yes, as
CommonMark reads it as H2) and whether type tag names can be slugs (recommended: yes, and a
paragraph of only `[essay]` is always the tag).

## Evidence

- `docs/question-types/essay.md:30-36`: Essay grammar does not say it applies to a paragraph,
so it accepts the content of a list item

  ```markdown
  The body consists of a single `[essay]` tag.
  ```
- `docs/question-types/essay.md:59-61`: The answer key section has no grammar, and its heading
variants and extent are not specified

  ```markdown
  It starts with an `H2` heading with the text `## [answer-key]`
  ```
- `docs/question-types/essay.md:92-95`: An empty or repeated answer key section has no
documented rule

  ```markdown
  | answerKey | error    | blank-text                  | if defined, must have at least one
visible character       |
  ```
- `docs/question-types/essay.md:53-78`: The text never maps the answer key section to the
`answerKey` field

  ```markdown
  ## Answer key
  ```
- `docs/question-types/question-base.md:194`: SLUG accepts `essay`, so `[essay]` at the start
of the first paragraph is a slug

  ```lark
  SLUG: /[a-zA-Z0-9]+([-_][a-zA-Z0-9]+)*/
  ```
- `docs/question-types/essay.md:59-61`: Essay spec does not say how footnotes resolve across
the stem, epilogue and answer key

  ```markdown
  The answer key section is placed below the `[essay]` tag and any existing
  ```

## Impact

- **Essay grammar does not say it applies to a paragraph, so it accepts the content of a list
item:** The block context of the tag comes from implementation behavior and a footnote, not
from the grammar.
- **The answer key section has no grammar, and its heading variants and extent are not
specified:** Two implementations can split the same document into a different epilogue and
answer key.
- **An empty or repeated answer key section has no documented rule:** The corpus enforces a
rule the spec does not state, and an empty section, which is almost always an unfinished
question, passes with only the `missing-answer-key` info at most.
- **The text never maps the answer key section to the `answerKey` field:** A reader of the
Markdown spec cannot tell which field holds the answer key, or that it is a field of the body
````

```
and therefore invalid in the frontmatter (`invalid-frontmatter-field`).
- **SLUG accepts `essay`, so `[essay]` at the start of the first paragraph is a slug:**
Contrived inputs where the spec gives two readings of the same bracket.
- **Essay spec does not say how footnotes resolve across the stem, epilogue and answer key:**
The student view renders a dangling footnote, or a host that renders the whole document shows
text from the answer key.

## Suggested fix

- **Essay grammar does not say it applies to a paragraph, so it accepts the content of a list
item:** State that the body is a paragraph whose only content is the rule, for example
`essay_block : ws? "[essay]" ws? nl`, and that a list item or heading is not the tag. State
that a later `[essay]` paragraph is a forbidden element of the epilogue, and list the parse
errors in the Additional Rules table, as other types do.
- **The answer key section has no grammar, and its heading variants and extent are not
specified:** Add a grammar rule, for example `answer_key : "##" ws "[" "answer-key" "]" ws? nl
md_block+`. State that the heading is an ATX H2 alone on its line (decide on the setext form;
CommonMark reads both as H2), and that the section holds every block after it, headings
included except H1, up to the end of the question (end of file, or the next block in an exam).
- **An empty or repeated answer key section has no documented rule:** Add a rule for a second
section (`parse-error`) to the table. For the empty section, two readings are defensible: (a)
an empty section is an absent answer key, reported only as `missing-answer-key`, as the corpus
decides (examples/valid/essay/empty-answer-key.mdq.md); (b) it is a defined but blank field,
`blank-text`, as for `title:` with only spaces. Recommended: (a), since `## [answer-key]` with
nothing after it is a common template placeholder, and state it in the text and the table.
- **The text never maps the answer key section to the `answerKey` field:** Add one sentence:
the blocks of the section, without the heading, are the `answerKey` string, with the same
normalization license as `stem` and `epilogue`. Add the YAML form of the example.
- **SLUG accepts `essay`, so `[essay]` at the start of the first paragraph is a slug:** State
whether a type tag name such as `essay` can be a slug, and that a paragraph of only `[essay]`
is always the essay tag.
- **Essay spec does not say how footnotes resolve across the stem, epilogue and answer key:**
State that a footnote definition belongs to the field it is in, and lint a reference defined in
 another field.

## Acceptance criteria

- [ ] essay.md has grammar rules for the tag paragraph and the answer key section
- [ ] The text states the extent of the section and that it fills `answerKey`
- [ ] The Additional Rules table lists the second-section and tag-paragraph errors
- [ ] The corpus has examples for a section with headings inside and for the empty section
- [ ] scripts/spec_audit.py passes

--- END ISSUE 1 ---
```

````
--- ISSUE 2 ---
# [Spec] Define the response format of a "text" essay

**Labels:** high

## Problem

`input: text` is a rich text editor, but neither essay.md nor responses.md says whether the
response string is Markdown, HTML or plain text. Options: (a) CommonMark source for `text`,
verbatim text for `plain` and `code`; (b) leave it to the host. Recommended: (a), since MDQ is
Markdown and a host with an HTML editor can convert.

## Evidence

- `docs/question-types/essay.md:47-50, docs/responses.md:38`: The response format of a "text"
essay is not defined

  ```markdown
  "text" indicates that the response should be a rich text input, potentially
  supporting markdown or other formatting.
  ```

## Impact

Two hosts that exchange responses (an LMS, a grader, an exporter) can store the same answer as
Markdown in one and as HTML in the other. A renderer cannot tell which it received, and raw
HTML in a response is also an injection risk when the host renders it as Markdown.

## Suggested fix

State in `docs/responses.md` and in Input that the response of every essay is a string, and
that for `input: text` it is CommonMark source. A host whose editor produces HTML converts it
before it stores the response. `plain` and `code` responses are verbatim text.

## Acceptance criteria

- [ ] responses.md and essay.md state the format of the response for each `input`
- [ ] A corpus response example exists for an essay

--- END ISSUE 2 ---
````

````
--- ISSUE 3 ---
# [Spec] Document essay grading, response and answer key visibility in essay.md

**Labels:** medium, low, info

## Problem

The essay Grading section is one sentence. The automation, the pending state, the [0, 1] score
range, the exam `grading` exclusion and the response shape live in three other documents, and
the text does not say when the answer key can be shown. Recommended: add Automation and
Response sections as ordering has, and state that the answer key MUST NOT be shown before the
student submits and MAY be shown after, even while pending. Also decide whether MDQ covers
response length limits (recommended: no; state that they go in the stem or `meta`).

## Evidence

- `docs/question-types/essay.md:80-82`: The Grading section does not give the score range,
automation or response

  ```markdown
  Essay questions are graded manually.
  ```
- `docs/question-types/essay.md:55-57`: When the answer key may be shown to students is not
defined

  ```markdown
  key section is optional and serves as a guide for the instructor to grade the
  responses and may be presented as feedback to the students.
  ```
- `docs/question-types/essay.md:15-22`: No fields for response size or attachments

  ```markdown
  | input     | string  | Either "code", "text" or "plain"[^2]      |
  ```

## Impact

- **The Grading section does not give the score range, automation or response:** A reader of
the essay document cannot learn how an essay is scored or what its response looks like without
reading three other documents.
- **When the answer key may be shown to students is not defined:** A host can show the model
answer while the exam is open, and the rule that a pending response shows no feedback seems to
forbid showing it at all until the instructor grades.
- **No fields for response size or attachments:** Authors write length limits in the stem
(`essay-tag-with-text` shows the attempt), where no host can enforce them.

## Suggested fix

- **The Grading section does not give the score range, automation or response:** Expand
Grading: the automation is `manual`, every response is pending until the instructor gives a
score in [0, 1], the exam `grading` does not apply and `penalty` does. Add a Response section
that links to `responses.md`.
- **When the answer key may be shown to students is not defined:** State that the answer key
MUST NOT be shown to the student before they submit the response, and that it MAY be shown
after, even while the response is pending.
- **No fields for response size or attachments:** Decide whether MDQ covers these. If yes, add
optional `minWords`/`maxWords` fields; if no, state that limits belong in the stem or in
`meta`.

## Acceptance criteria

- [ ] essay.md has Grading, Automation and Response sections with links to responses.md and
exam.md
- [ ] The text states when the answer key may be shown

--- END ISSUE 3 ---
````

````
--- ISSUE 4 ---
# [Spec] Use one placement for `## [name]` sections relative to the epilogue

**Labels:** medium

## Problem

Essay puts its answer key after the epilogue; ordering puts its sections right after the body
and the epilogue after them. The exam rules about `---` and setext headings name only the
epilogue, although mdq-py applies them to the answer key. Options: (a) sections always after
the epilogue, as question-base.md describes; (b) sections always right after the body.
Recommended: (a), since it agrees with question-base.md and the essay corpus; ordering then
follows in its own audit. Extend the exam rules to every block after the body.

## Evidence

- `docs/question-types/essay.md:59-60, docs/question-types/ordering.md:166-168`: Essay places
its answer key section after the epilogue, ordering places its sections before it

  ```markdown
  The answer key section is placed below the `[essay]` tag and any existing
  epilogue block.
  ```
- `docs/exam.md:161-165`: Exam rules about `---` and setext headings do not cover the essay
answer key

  ```markdown
  Inside an exam, the epilogue of a question MUST NOT use `---` as a thematic
  break.
  ```

## Impact

- **Essay places its answer key section after the epilogue, ordering places its sections before
 it:** Authors must learn two placements for the same `## [name]` construct, and a parser
cannot share the section logic.
- **Exam rules about `---` and setext headings do not cover the essay answer key:**
Implementations can accept a `---` in an answer key, and a later question then starts inside
it.

## Suggested fix

- **Essay places its answer key section after the epilogue, ordering places its sections before
 it:** Choose one placement for `## [name]` sections relative to the epilogue and use it in
question-base.md, essay.md and ordering.md.
- **Exam rules about `---` and setext headings do not cover the essay answer key:** Extend the
exam rules to every block after the body, the answer key included.

## Acceptance criteria

- [ ] question-base.md, essay.md and ordering.md state the same placement
- [ ] exam.md applies the `---` and setext rules to the answer key
- [ ] A corpus exam example has an essay with an answer key

--- END ISSUE 4 ---
````

````
--- ISSUE 5 ---
# [Spec] Use one field name for the code/text choice of essay and ordering

**Labels:** medium

## Problem

Essay `input` and ordering `content` select the same `code`/`text` mode, with the same
`highlight` rules. Options: (a) keep both and document why (essay describes the editor,
ordering describes the lines); (b) rename one. Recommended: (a), since the meanings differ
(`plain` exists only for an editor) and a rename breaks documents; add one sentence to each
document.

## Evidence

- `docs/question-types/essay.md:21, docs/question-types/ordering.md:98`: Essay `input` and
ordering `content` hold the same code/text choice under different names

  ```markdown
  | input     | string  | Either "code", "text" or "plain"[^2]      |
  ```

## Impact

Authors who write both types must remember two names for one concept.

## Suggested fix

Use one field name for the code/text choice in both types, or document why they differ.

## Acceptance criteria

- [ ] essay.md and ordering.md explain the relation between `input` and `content`, or one field
 is renamed with a changelog entry

--- END ISSUE 5 ---
````

````
--- ISSUE 6 ---
# Mechanical fixes

**Labels:** mechanical, medium, low

## Problem

Each fix below follows from its finding and needs no design decision, so they can be applied
together in one pass and reviewed by reading the diff. Severity still applies: a mechanical fix
 can matter a lot.

### F1 (medium): Essay grammar rejects an indented [essay] tag and trailing spaces

`docs/question-types/essay.md:35`

```lark
essay : "[essay]"
```

**Suggested fix:** Write the rule as `essay : ws? "[essay]" ws?`, as the numeric and short-
answer rules do.

### F11 (low): A warning rule is written as "must"

`docs/question-types/essay.md:24, docs/question-types/essay.md:91-94, docs/question-
types/ordering.md:339`

```markdown
| highlight | warning  | ignored-highlight            | must be omitted unless `input` is
"code"                   |
```

**Suggested fix:** Write "should be omitted unless ..." in both tables, and in footnote 1 add
that the property SHOULD be omitted.

### F20 (low): Essay spec says both that essays may require and that they always use manual
grading

`docs/question-types/essay.md:55, docs/question-types/essay.md:82`

```markdown
Essay questions are usually open-ended, and may require manual grading.
```

**Suggested fix:** Replace "may require manual grading" with a statement that every essay
response is graded by the instructor.

### F28 (low): schema/essay.yaml gives `highlight` a validation role the spec does not have

`schema/essay.yaml:32-33`

```yaml
Programming language used for syntax highlighting and, where
          supported, validation.
```

**Suggested fix:** Remove the validation clause from the schema, or describe the behavior in
essay.md.

### F29 (low): schema/essay.yaml rejects an empty `highlight`, the spec does not say so

`schema/essay.yaml:35`

```yaml
minLength: 1
```

**Suggested fix:** Add the constraint to the essay frontmatter table or footnote [^1].

## Acceptance criteria
````

```
- [ ] F1: Essay grammar rejects an indented [essay] tag and trailing spaces
- [ ] F11: A warning rule is written as "must"
- [ ] F20: Essay spec says both that essays may require and that they always use manual grading
- [ ] F28: schema/essay.yaml gives `highlight` a validation role the spec does not have
- [ ] F29: schema/essay.yaml rejects an empty `highlight`, the spec does not say so

--- END ISSUE 6 ---
```